

Let's examine this. Earlier, I said that to make a call to a method of an object, you use the following generic syntax:

```
<object>.<method>(<any arguments>)
```

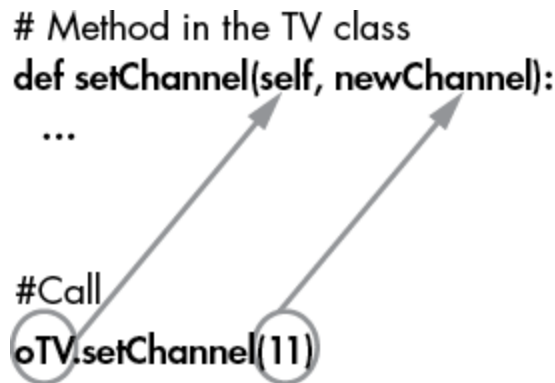
Python takes the `<object>` you specify in the call and rearranges it to become the first argument. Any values in the parentheses of the method call are considered the subsequent argument(s). Thus, Python makes it appear that you wrote this instead:

```
<method of object>(<object>, <any arguments>)
```

[Figure 2-5](#) shows how this works in our example code, again using the `setChannel()` method of the `TV` class.

```
# Method in the TV class
def setChannel(self, newChannel):
    ...

#Call
oTV.setChannel(11)
```



[Figure 2-5](#): Calling a method

Although it looks like we're only providing one argument here (for `newChannel`), there are really two arguments passed in—`oTV` and `11`—and the method provides two parameters to receive these values (`self` and `newChannel`, respectively). Python rearranges the arguments for us when the call is made. This may seem odd at first, but it will become second nature very quickly. Writing the call with the object first makes it much easier for a programmer to see which object is being acted on.

This is a subtle but important feature. Remember that the object (in this case, `oTV`) keeps the current settings of all of its instance variables. Passing the object as the first argument allows the method to run with the values of the instance variables of that object.